

CSC241 – Object Oriented Programming (Java)
THEORY ASSIGNMENT 1

Object Oriented Analysis of a Smart Campus Allocation System

Additional task = Last registration digit = 5 → Emergency continuity

Submitted by: Ayesha

Registration ID: SP26-BAI-055

Submitted to: Dr Muhammad Shahid Bhatti

Submission date: 25 Sep 2026



Task 1 – Stakeholder and Requirement Analysis

The main stakeholders and their needs are:

Stakeholder	Their need
Students	Students need a suitable room for group study, assignments, or meetings.
Faculty members	Teachers need proper rooms for their classes and academic activities.
Administrators	Administration needs to manage rooms properly and avoid unnecessary empty rooms.
Security staff	Security staff need records to know who used a room and when.

Requirement conflicts

1. Student vs faculty: Students need fair access, while faculty may need priority for teaching and urgent academic work

Assumption: I assume scheduled teaching gets priority when two valid requests conflict.

2. Room utilization vs safety: The administration wants maximum use of rooms, but a damaged room may need to be closed by maintenance staff.

Assumption: Safety should come first. A room marked unsafe should not be open.

3. Confirmed reservations vs emergency continuity: A student may have already booked a room, but an emergency such as a building closure can make the room unavailable.

Assumption: The booking may be cancelled or moved to another suitable room. The user should be informed about the change.

Facts and Assumptions

Facts given in the scenario:

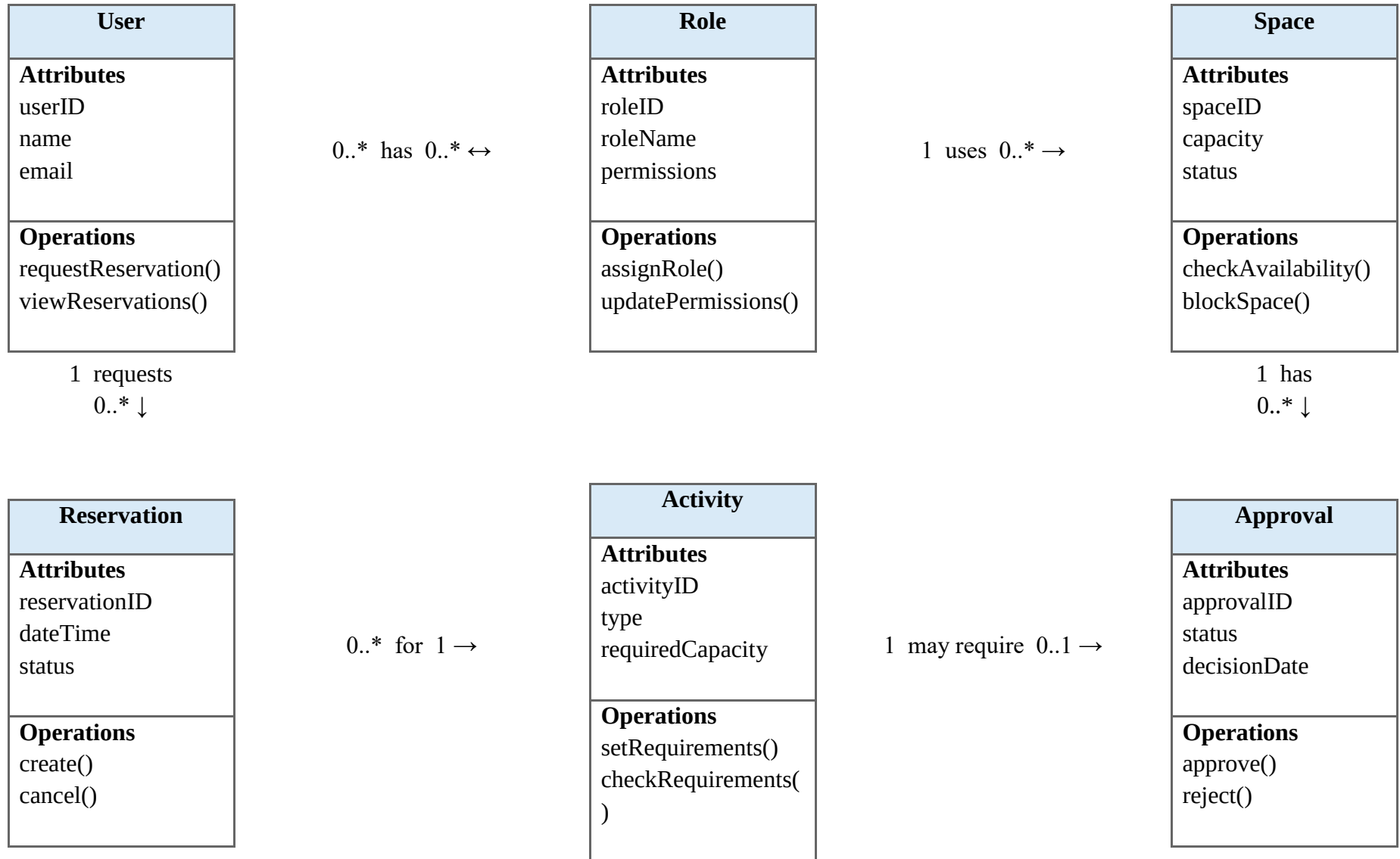
- The system will manage different campus spaces.
- Students, teachers, administration, security staff are involved.
- Different activities have different room requirements.
- A person can have more than one role.
- Emergencies can affect confirmed bookings.
- Some rules have not yet been decided.

My assumptions:

- Teachers get priority for scheduled classes.
- Unsafe rooms cannot be booked.
- During an emergency, an authorized administrator can cancel or move bookings.
- Users should be informed if their booking is changed.

Task 2 – Object Oriented Conceptual Model

Nine classes are proposed. Each class has one main responsibility and keeps related operations together.



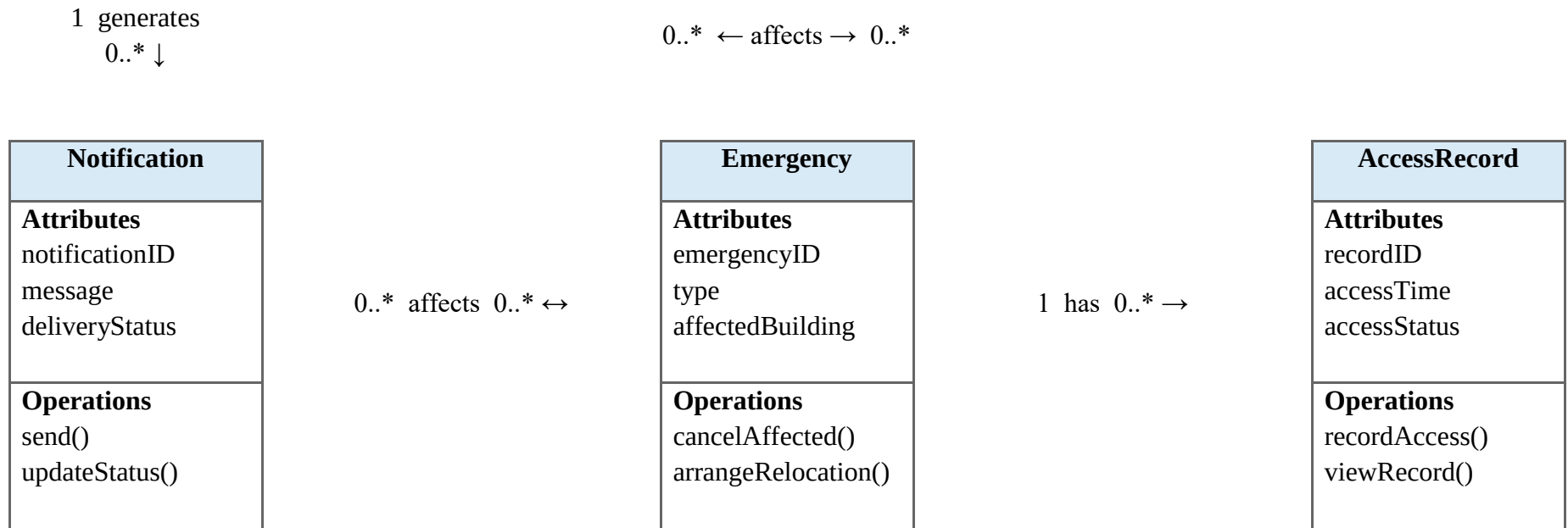


Figure 1. UML class diagram. Multiplicities are shown on each association; arrows show the main direction of use.

Class responsibilities

Class	Responsibility	attributes / operations
User	Stores user identity and lets a user request or view reservations.	userID, name; requestReservation(), viewReservations()
Role	Stores roles and permissions because one person may have more than one role.	roleID, roleName; assignRole(), updatePermissions()
Space	Stores room status and checks whether a room can be used.	spaceID, capacity; checkAvailability(), blockSpace()
Reservation	Controls the booking record and its current status.	reservationID, dateTime; create(), cancel()
Activity	Describes why the space is needed and its capacity requirements.	activityID, type; setRequirements(), checkRequirements()
Approval	Keeps the approval decision separate from the reservation itself.	approvalID, status; approve(), reject()
Emergency	Handles building closure, cancellation and relocation of affected reservations.	emergencyID, type; cancelAffected(), arrangeRelocation()
Notification	Sends updates to users when reservation status changes.	notificationID, message; send(), updateStatus()

AccessRecord	Keeps traceable access information for security and audit.	recordID, accessTime; recordAccess(), viewRecord()
--------------	--	--

Responsibility separation

I have divided the work between different classes instead of putting everything in one class. For example, **User** only represents the person, while **Reservation** handles booking information. **Space** checks whether a room is available, and **Approval** handles permission for bookings.

Similarly, **Emergency** deals with emergency situations and **Notification** informs users about changes. This makes the system easier to understand and modify. If the notification method changes, I only need to update the **Notification** class instead of changing the whole system.

Personal example: If I book a room for a group assignment, the **User** represents me, **Reservation** stores my booking, **Space** represents the room, and **Notification** can inform me if the room is changed because of an emergency.

Task 3 – Abstraction, Encapsulation and Invariants

User: Abstracts a real campus person. Email and identity details should be hidden. They are accessed through controlled operations such as requestReservation() and viewReservations().

Space: Abstracts a physical room. Capacity and safety/status should be protected so users cannot directly change them. Operations such as checkAvailability() and blockSpace() control changes.

Reservation: Abstracts a booking between a user, an activity and a space. Status and date/time should be protected and changed through create() and cancel() or other authorized operations.

Invariants

1. A reservation cannot be confirmed for a blocked or unsafe space.
2. A reservation must have one requesting user and one allocated space.
3. The reserved space must have enough capacity for the activity.
4. A cancelled reservation cannot become active again without a new valid approval process.
5. During an emergency closure, affected confirmed reservations must be cancelled or relocated and affected users must be notified.

Encapsulation helps because important data is changed through operations instead of being directly edited. This reduces the chance of creating an invalid reservation state.

Task 4 – Alternative Designs and Decision

Option 1: Inheritance

Use separate classes like Student, Teacher, and Administrator inherited from User.

Option 2: Composition

Use one User class with separate Role objects. Roles can be added or removed when needed.

Criteria	Inheritance	Composition
Multiple roles	Difficult	Easy
Changing permissions	Difficult	Easy
Extensibility	Moderate	High

Design chosen:

I choose **composition** because one person can have multiple roles and roles can change during the semester.

Task 5 – Behaviour and Edge Cases

State	Event	Next state
Requested	request submitted	Pending Approval
Pending Approval	approved	Confirmed
Pending Approval	rejected	Rejected
Confirmed	cancelled	Cancelled
Confirmed	emergency closure	Relocated or Cancelled

Figure 2. Reservation state transition table.

Two simultaneous requests: Reservation checks availability and approval rules. Only one request can be confirmed for the same space and time; the other remains pending or is rejected.

Maintenance block after confirmation: Space changes to blocked status. The system checks affected confirmed reservations. If the room cannot be used, Reservation is cancelled or relocated and Notification informs the affected user.

Emergency cancellation or relocation: Emergency identifies the affected building and reservations. It triggers cancellation or relocation, while Notification informs affected users and the record is retained for audit.

Decision enforcing:

- **AllocationService:** Handles booking conflicts.
- **Space:** Controls room availability and maintenance blocks.
- **Emergency:** Handles emergency decisions.
- **Notification:** Informs affected users.

Task 6 – Independent Exploration: Role-Based Access Control

Role-Based Access Control (RBAC) means giving permissions to users according to their roles instead of giving every user separate permissions. For example, a student can request a room, while an administrator can approve or cancel reservations. A user can also have more than one role. OWASP explains RBAC as a model where permissions are connected to roles and users receive those permissions through their assigned roles.

In my **Smart Campus Allocation System**, RBAC improves the User and Role classes. For example, if I am a **student and teaching assistant**, I can have both roles. The Role object can store permissions such as `requestSpace()` or `approveReservation()`. This is better than creating separate classes for every combination of roles because roles can change easily during the semester.

RBAC also helps protect important operations. For example, an ordinary student should not be able to block a room or cancel another person's reservation. Only users with the required role should be allowed to perform these actions. This keeps responsibilities clear and improves security.

AI Use Disclosure

Generative AI was used to assist with organizing and drafting parts of this assignment. I reviewed and revised the content and take responsibility for the final design decisions and submitted work.

References

1. Association for Computing Machinery and IEEE Computer Society. (2020). Computing Curricula 2020: Paradigms for Global Computing Education. <https://www.acm.org/education/curricula-recommendations>
2. Seoul Accord. Graduate Attributes, Section D.4.1 Range of Problem Solving.
- 3.OWASP Foundation. *Authorization Cheat Sheet*. [OWASP Authorization Cheat Sheet](#)
- 4.OWASP Foundation. *C1: Implement Access Control*. [OWASP Top 10 Proactive Controls](#)